HDOC DAY – 56

ALGORITHM:

Algorithm: first_negative()

1. Start.
2. Read the size of the array.
3. Declare a variable-length integer array arr[size].
4. Read the elements of the array.
5. Read the window size k.
6. Pass the address of the array to first_negative().
7. Check whether k is valid:
    - If k <= 0 or k > size, print -1 and return.
8. Use an outer loop from i = 0 to size - k to create each sliding window.
9. For each window:
    - Set found = 0.
    - Use an inner loop from j = i to i + k - 1.
    - Check whether *(arr + j) < 0.
    - If true, print that element, set found = 1, and use break because the first negative has been found.
    - If no negative element is found, print 0.
10. Separate each result using a space.
11. Stop.

C PROGRAM:

```
#include <stdio.h>


void first_negative(int *arr, int size, int k)
```

```
{
    int i, j;
    int found;

    // Check if k is invalid
    if (k <= 0 || k > size)
    {
        printf("-1\n");
        return;
    }

    // Move the sliding window from left to right
    for (i = 0; i <= size - k; i++)
    {
        found = 0;

        // Check each element of the current window
        for (j = i; j < i + k; j++)
        {
            if (*(arr + j) < 0)
            {
                printf("%d", *(arr + j));
                found = 1;
                break;   // First negative found
            }
```

```
        }

        // If no negative number is found
        if (found == 0)
        {
            printf("0");
        }

        // Print space except after the last result
        if (i < size - k)
        {
            printf(" ");
        }
    }

    printf("\n");
}

int main()
{
    int size, i, k;

    printf("Enter the size of array: ");
    scanf("%d", &size);
```

```
    int arr[size];   // Variable-length array

    printf("Enter the array elements: ");
    for (i = 0; i < size; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Enter k: ");
    scanf("%d", &k);

    // Pass the address of the array
    first_negative(arr, size, k);

    return 0;
}
```

TEST CASE TABLE:

Test Case	Array	k	Expected Output
1	12 -1 -7 8 -15 30 16 28	3	-1 -1 -7 -15 -15 0
2	1 2 -3 4 5 -6	2	0 -3 -3 0 -6
3	1 2 3 4	2	0 0 0
4	1 2 3	4	-1
5	-5 2 -3 4	1	-5 0 -3 0